

DHANNYA

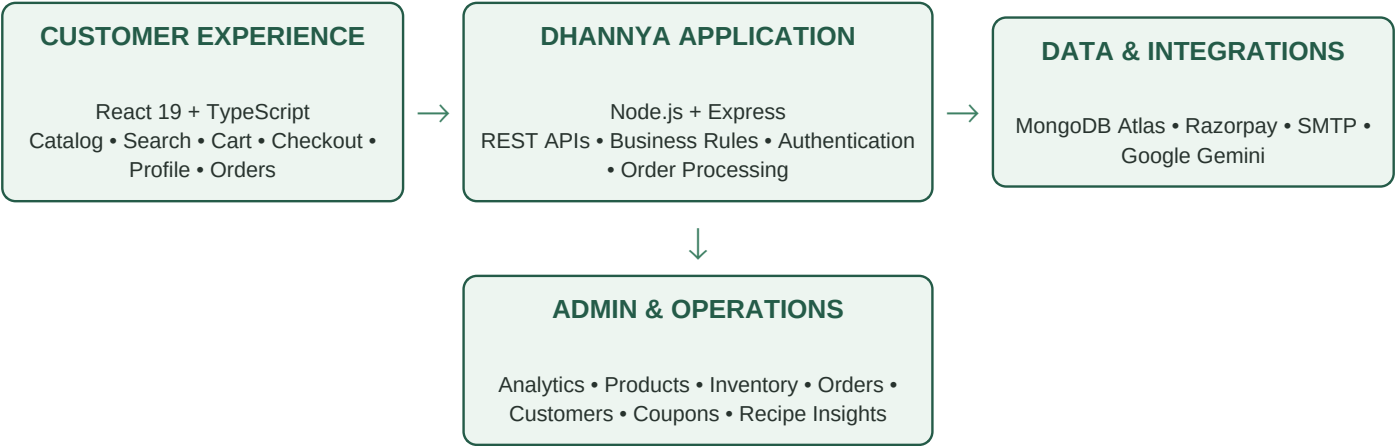
Organic & Custom Masala Store

Full System Architecture & Technical Explanation

Architecture purpose

Dhannya is a full-stack e-commerce platform designed for selling organic masalas and supporting personalized spice-blend creation. The architecture separates the customer experience, application services, persistent data, external integrations, and administration so the platform can evolve from an initial product into a scalable commerce system.

Executive Architecture

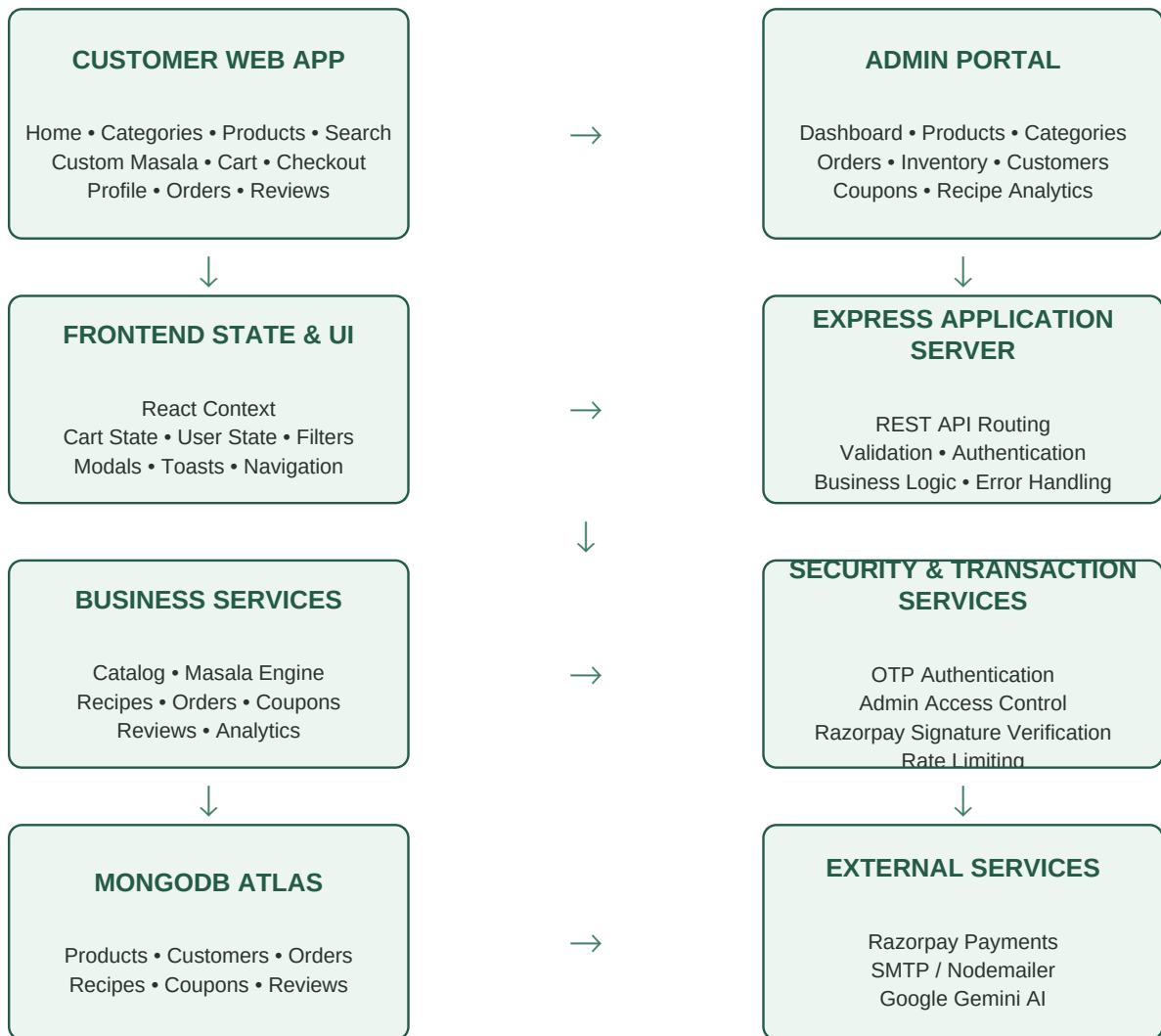


Key Architectural Principles

- **Modular**: Customer, admin, catalog, masala, orders, payments and notifications are logically separated.
- **API-driven**: The frontend communicates with backend services through REST APIs.
- **Data-centric**: MongoDB Atlas provides persistent storage for products, customers, orders, recipes, coupons and reviews.
- **Integration-ready**: Razorpay, SMTP email and Google Gemini are isolated external services.
- **Operationally scalable**: Business logic is centralized in the backend, making future mobile clients and additional frontends easier to support.

1. Detailed System Architecture

The system follows a layered architecture. Requests originate from the customer or administrator interfaces, pass through the application server, and are then handled by the relevant business service or integration. Persistent information is stored in MongoDB Atlas.



How the layers work

- **Presentation:** React components provide the storefront and administrator experience.
- **State:** AppContext coordinates shared cart, authentication, search/filter state, active views, notifications and modal visibility.
- **Application:** Express exposes REST endpoints and coordinates authentication, catalog, custom-masala calculations, orders, coupons, reviews, analytics and payments.
- **Data:** Mongoose models map application objects to MongoDB collections.
- **Integrations:** Payment, email and AI capabilities are kept outside the core database layer.

2. Frontend & Backend Architecture

Frontend — Customer Experience

Component Area	Responsibility
Application Shell	App.tsx, navigation, route/view switching, overlay handling and mobile back-button behavior.
Storefront	Hero, categories, product listings, best sellers and product quick views.
Shopping	Cart drawer, checkout flow, product variants, coupon application and order creation.
Authentication	Email OTP workflow and customer profile/session state.
Custom Masala	Ingredient selection, proportion controls, roast preference and live calculation results.
Administration	KPI dashboard, orders, products, inventory alerts, customers, coupons and recipe analytics.

Backend — Core Application Services

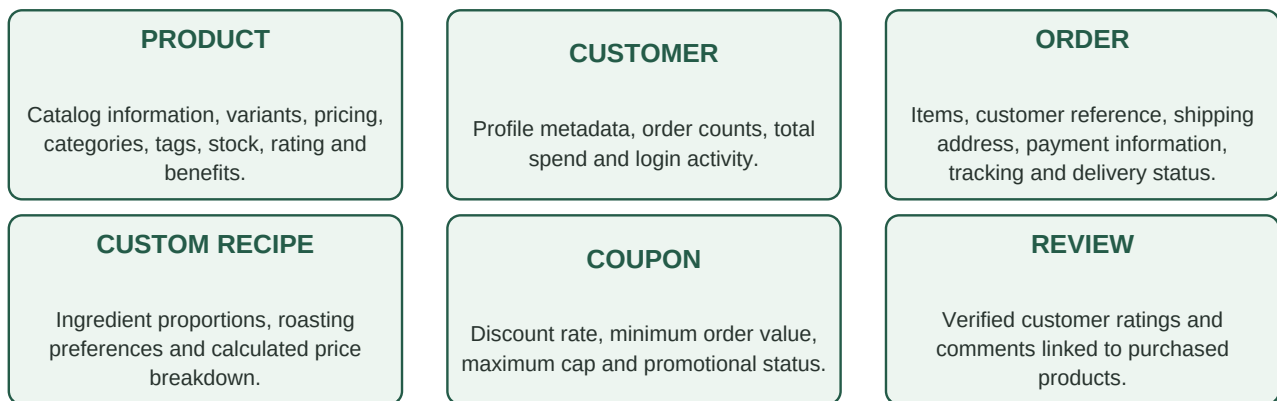
Service	Role in the platform
Authentication	OTP dispatch and verification, email checks, admin authentication and access protection.
Catalog	Products and categories, filtering, searching, sorting and product management.
Masala Engine	Ingredient availability, weight calculation, roasting charges and final price computation.
Recipes	Stores custom spice-blend definitions and price breakdowns.
Orders	Creates orders, generates tracking identifiers and triggers order notifications.
Coupons	Validates coupon eligibility, discount rules, minimum values and maximum discount caps.
Reviews	Handles verified customer ratings and comments.
Analytics	Provides KPIs, revenue views, stock alerts, sales trends and operational metrics.
Payments	Creates Razorpay orders and verifies payment signatures before confirming transactions.

3. API & Data Architecture

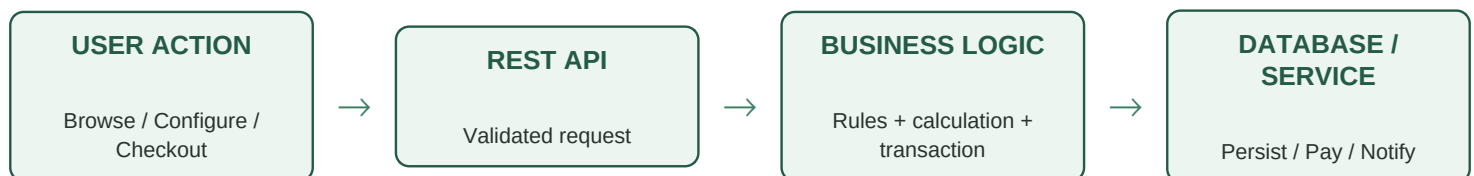
REST API Surface

Endpoint Group	Purpose
/api/health, /api/db-status	Health checks and MongoDB connection status.
/api/auth/*	OTP dispatch, verification, email checks and administrator authentication.
/api/products, /api/categories	Catalog retrieval, search, filtering and sorting.
/api/admin/products, /api/admin/categories	Administrator product and category CRUD operations.
/api/masalas/*	Ingredient listing and custom masala calculations.
/api/recipes	Create and retrieve custom spice recipes.
/api/orders	Order creation, tracking ID generation and notification workflow.
/api/coupons/*	Coupon retrieval and validation.
/api/payment/*	Razorpay order creation and payment signature verification.
/api/reviews	Verified product ratings and comments.
/api/admin/analytics	KPIs, revenue, stock alerts and sales analytics.

Database Model Architecture

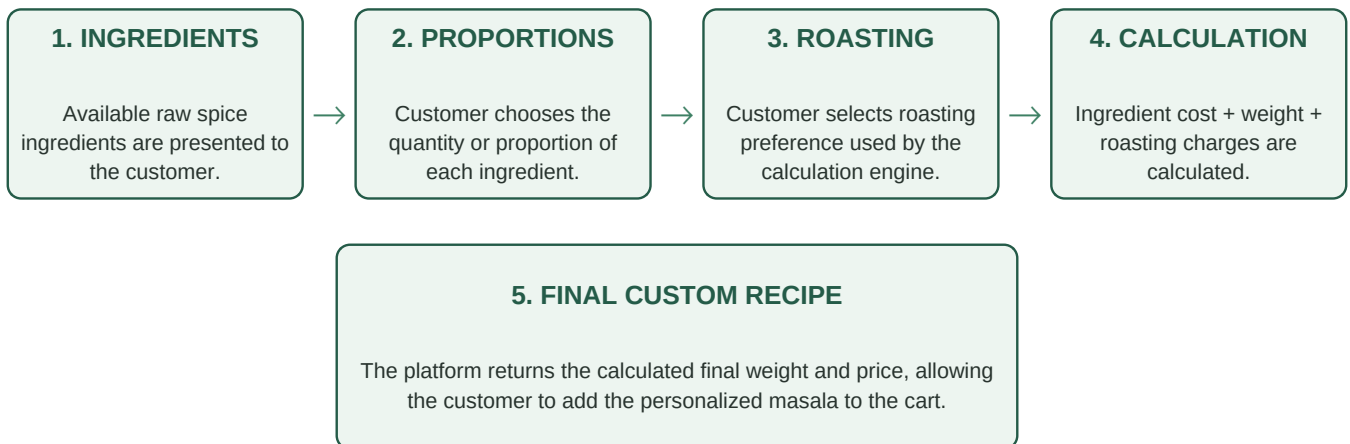


Data Flow

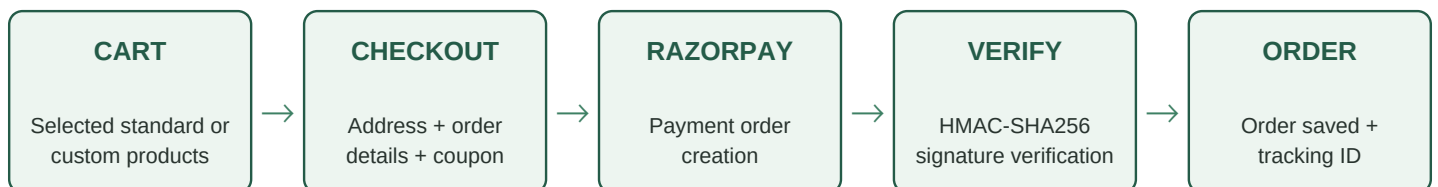


4. Custom Masala, Order & Payment Flows

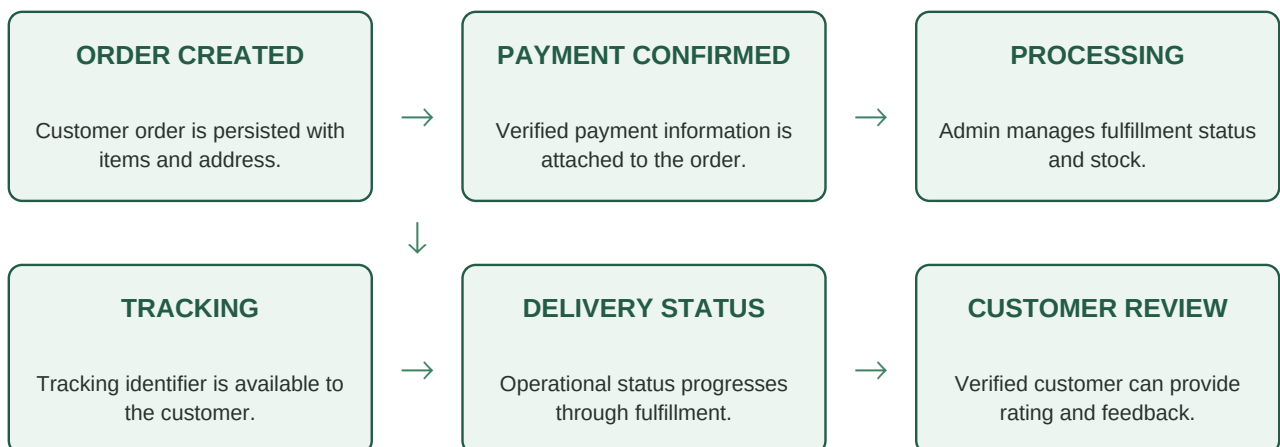
Custom Masala Engine



Checkout & Payment Flow



Order Lifecycle

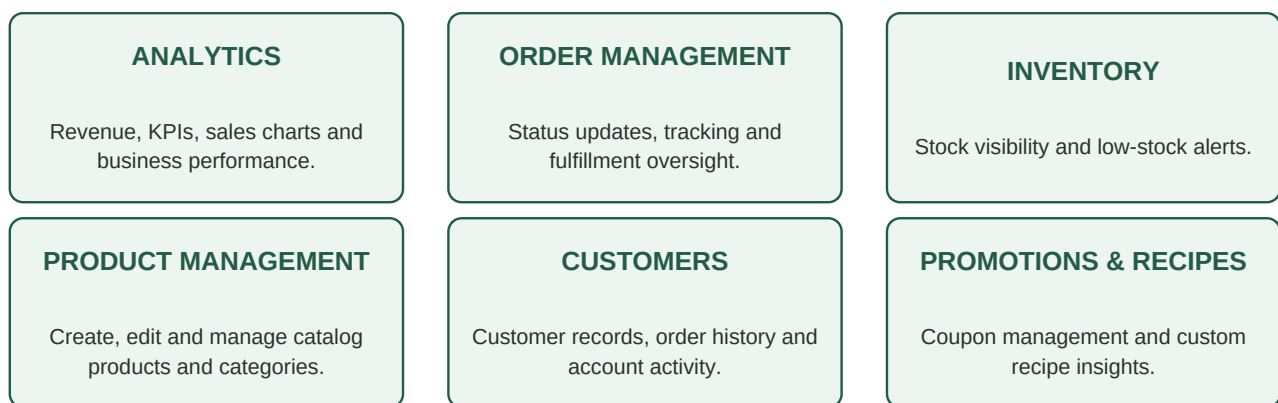


5. Security, Administration & Deployment

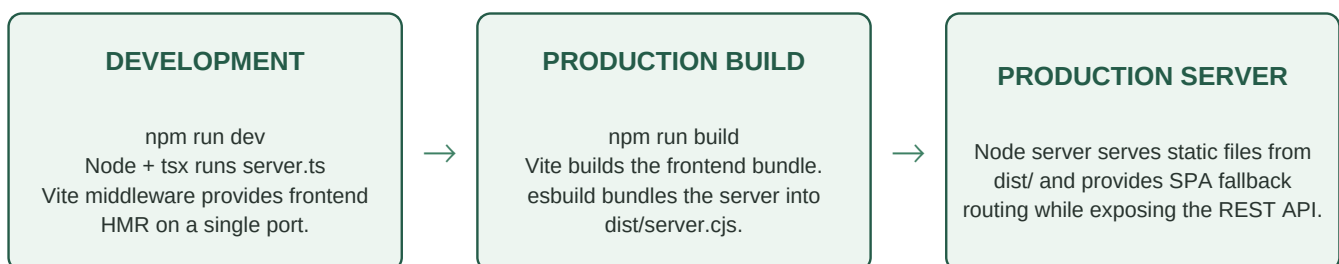
Security Architecture

Area	Approach	Purpose
Customer Authentication	Email OTP	Passwordless login and identity verification.
Administrator Access	Protected admin authentication + SHA-256 comparison	Restricts access to operational controls.
Payments	Razorpay signature verification	Confirms that payment responses are authentic before transaction completion.
Abuse Protection	Rate limiting / controlled OTP workflow	Reduces repeated authentication abuse.
Data Validation	Backend validation and business rules	Prevents invalid order, coupon and calculation data from reaching persistence.

Admin & Operations Architecture



Build & Deployment Architecture



6. Scalability & Investor Summary

Why this architecture can scale

Dimension	Current Architecture	Future Scaling Path
Frontend	React + Vite SPA	Mobile app, PWA, additional customer/admin clients using the same API layer.
Backend	Node.js + Express service	Horizontal server scaling and gradual service separation as traffic grows.
Database	MongoDB Atlas + Mongoose	Managed scaling, indexing, replication and performance optimization.
Payments	Razorpay integration	Additional payment methods or gateways can be added behind payment service boundaries.
Notifications	SMTP / Nodemailer	Transactional email provider expansion, SMS and WhatsApp notification integrations.
AI	Google Gemini SDK	AI-assisted recommendations, recipe guidance and customer support workflows.
Analytics	Admin analytics endpoints	Dedicated reporting warehouse or BI layer when data volume and reporting needs increase.

Investor-Level System Summary

CUSTOMER VALUE A digital storefront combines standard organic products with personalized masala creation.	TECHNICAL FOUNDATION Modern React frontend, Node/Express backend and managed MongoDB data layer.	TRANSACTION ENGINE Integrated payment verification, order processing, coupons and notifications.
OPERATIONS Admin tools provide visibility across products, inventory, customers, orders and performance.	DIFFERENTIATION Custom recipe calculation turns spice personalization into a core product capability.	GROWTH READY API-first structure supports future mobile clients, integrations, analytics and AI capabilities.
Architecture conclusion Dhannya is structured as a complete commerce platform rather than a simple storefront. The customer interface, centralized application services, persistent data, transaction integrations and administration layer form a clear foundation for launching the product and expanding its capabilities as customer volume, catalog size and operational complexity increase.		